太棒的想法！加入定時功能（Timer）能讓這個控制面板實用許多。
為了達到「網頁設定秒數 $\rightarrow$ 開啟開關 $\rightarrow$ Node-RED 倒數計時 $\rightarrow$ 時間到自動關閉燈號與網頁開關」的完整連動，我們需要做兩個重要的升級：

1. 網頁端：加入拉桿（Slider）介面，並在送出開啟指令時，一併將「秒數」傳送給後端。同時，網頁需要定時向後端查詢狀態（**Polling**），這樣時間一到，網頁上的開關才會自己「啪」一聲關起來。
2. **Node-RED** 後端：接收指令後啟動計時器（setTimeout），並將當前的燈號狀態儲存在全域變數中，供網頁隨時查詢。

以下是完整的實作步驟與設定。

系統架構解析

我們這次會建立 3 條 API 路由：

- **/web**：負責派發帶有拉桿的網頁介面。
- **/update**：接收網頁傳來的指令（例如 ?state=1&time=5），控制燈號並啟動計時。
- **/state**：專門讓網頁每秒詢問「現在燈還是亮著的嗎？」，以同步網頁開關狀態。

一鍵匯入完整流程碼

最快的方式是直接匯入我為您設計好的完整流程。請在 Node-RED 介面右上角選單點擊「匯入 (Import)」，將下方這段 JSON 程式碼貼上：

```
JSON
[
  {
    "id": "http_in_web_timer",
    "type": "http in",
    "z": "flow_workspace",
    "name": "網頁進入點",
    "url": "/web",
    "method": "get",
    "upload": false,
    "swaggerDoc": "",
    "x": 120,
    "y": 80,
    "wires": [
      [
        "template_html_timer"
      ]
    ]
  },
```

```
        "id": "template_html_timer",
        "type": "template",
        "z": "flow_workspace",
        "name": "HTML 計時器網頁",
        "field": "payload",
        "fieldType": "msg",
        "format": "html",
        "syntax": "mustache",
        "template": "<!DOCTYPE HTML>\n<html>\n<head>\n  <title>Node-RED Web Server</title>\n
<meta name=\"viewport\" content=\"width=device-width, initial-scale=1\">\n  <style>\n    html
{font-family: Arial; display: inline-block; text-align: center;}\n    h2 {font-size: 2.5rem; margin-bottom:
10px;}\n    .time-display {font-size: 2.0rem; margin: 10px 0; color: #333;}\n    body {max-width:
600px; margin:0px auto; padding-bottom: 25px;}\n    /* 拉桿樣式設計 */\n    .slidecontainer {width:
100%; margin: 25px 0;}\n    .slider-range {\n      -webkit-appearance: none; width: 70%; height: 12px;
border-radius: 5px;\n      background: #d3d3d3; outline: none; opacity: 0.8; transition: .2s;\n    }\n
.slider-range::-webkit-slider-thumb {\n      -webkit-appearance: none; appearance: none; width:
28px; height: 28px;\n      border-radius: 5px; background: #2b3a55; cursor: pointer;\n    }\n    /* 開關樣
式設計 */\n    .switch {position: relative; display: inline-block; width: 120px; height: 68px;}\n    .switch
input {display: none;}\n    .slider {position: absolute; top: 0; left: 0; right: 0; bottom: 0;
background-color: #ccc; border-radius: 34px;}\n    .slider:before {position: absolute; content: \"\";
height: 52px; width: 52px; left: 8px; bottom: 8px; background-color: #fff; -webkit-transition: .4s;
transition: .4s; border-radius: 68px;}\n    input:checked+.slider {background-color: #2196F3;}\n
input:checked+.slider:before {-webkit-transform: translateX(52px); -ms-transform:
translateX(52px); transform: translateX(52px);}\n  </style>\n</head>\n<body>\n  <h2>ESP Web
Server</h2>\n  \n  <div class=\"time-display\"><span id=\"timeValue\">5</span> s</div>\n  \n  <div
class=\"slidecontainer\">\n    \n    <input type=\"range\" min=\"1\" max=\"60\" value=\"5\"
class=\"slider-range\" id=\"timerSlider\" oninput=\"updateTime(this.value)\">\n  </div>\n\n  <label
class=\"switch\">\n    <input type=\"checkbox\" onchange=\"toggleCheckbox(this)\" id=\"output\">\n
<span class=\"slider\"></span>\n  </label>\n  \n  <script>\n    // 更新畫面上顯示的秒數\n    function
updateTime(val) {\n      document.getElementById(\"timeValue\").innerHTML = val;\n    }\n\n    // 切換開
關時，將狀態與設定的秒數送出\n    function toggleCheckbox(element) {\n      var xhr = new
XMLHttpRequest();\n      var time = document.getElementById(\"timerSlider\").value;\n      var
timestamp = new Date().getTime();\n      \n      if(element.checked) { \n        xhr.open(\"GET\",
\"/update?state=1&time=\" + time + \"&t=\" + timestamp, true); \n      } else { \n        xhr.open(\"GET\",
\"/update?state=0&t=\" + timestamp, true); \n      }\n      xhr.send();\n    }\n\n    // 每 1 秒向伺服器確認目
前的真實狀態 (確保時間到時開關會自動切回 off)\n    setInterval(function () {\n      var xhttp = new
XMLHttpRequest();\n      xhttp.onreadystatechange = function() {\n        if (this.readyState == 4 &&
this.status == 200) {\n          if(this.responseText === \"1\") {\n
document.getElementById(\"output\").checked = true;\n          } else {\n
document.getElementById(\"output\").checked = false;\n          }\n        }\n      };\n      xhttp.open(\"GET\",
\"/state?t=\" + new Date().getTime(), true);\n      xhttp.send();\n    }, 1000);\n
</script>\n</body>\n</html>",
        "output": "str",
        "x": 340,
        "y": 80,
        "wires":
```

```
        "http_out_web_timer"
    ]
  },
  {
    "id": "http_out_web_timer",
    "type": "http response",
    "z": "flow_workspace",
    "name": "回傳網頁",
    "statusCode": "",
    "headers": {},
    "x": 520,
    "y": 80,
    "wires": []
  },
  {
    "id": "http_in_update_timer",
    "type": "http in",
    "z": "flow_workspace",
    "name": "API 接收指令",
    "url": "/update",
    "method": "get",
    "upload": false,
    "swaggerDoc": "",
    "x": 130,
    "y": 180,
    "wires": [
      [
        "function_timer_logic"
      ]
    ]
  },
  {
    "id": "function_timer_logic",
    "type": "function",
    "z": "flow_workspace",
    "name": "計時與狀態控制",
    "func": "let reqState = msg.payload.state;\nlet reqTime = parseInt(msg.payload.time ||
\"0\");\n\n// 從記憶體取得目前的計時器ID\nlet timerId = context.get('timerId');\n\nif (reqState === \"1\")
{\n    // 紀錄當前狀態為開啟\n    global.set(\"ledState\", \"1\");\n    // 非同步輸出：回傳OK給網頁，並傳送
true 給 LED 亮燈\n    node.send([{ payload: \"OK\" }, { payload: true }]);\n\n    // 如果之前有還沒跑完的
計時器，先清除掉\n    if (timerId) clearTimeout(timerId);\n\n    // 啟動新的計時器\n    timerId =
```

```
setTimeout(function() {\n        global.set(\"ledState\", \"0\");\n        node.send([null, { payload: false }]);\n    }, reqTime * 1000);\n    \n    // 將新的計時器ID存入記憶體\n    context.set('timerId', timerId);\n} else {\n    // 手動關閉時，取消計時器\n    if (timerId) {\n        clearTimeout(timerId);\n        context.set('timerId', null);\n    }\n    global.set(\"ledState\", \"0\");\n    node.send([{ payload: \"OK\" }, { payload: false }]);\n}\n\n// 如果 Node-RED 重新部署，自動清除計時器防止記憶體洩漏\nnode.on(\"close\", function() {\n    let tid = context.get('timerId');\n    if (tid) clearTimeout(tid);\n});\n\nreturn null;"
        "outputs": 2,
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 320,
        "y": 180,
        "wires": [
            [
                "http_out_update_timer"
            ],
            [
                "ui_led_timer"
            ]
        ]
    },
    {
        "id": "http_out_update_timer",
        "type": "http response",
        "z": "flow_workspace",
        "name": "回傳 200 OK",
        "statusCode": "200",
        "headers": {},
        "x": 520,
        "y": 140,
        "wires": []
    },
    {
        "id": "ui_led_timer",
        "type": "ui_led",
        "z": "flow_workspace",
        "group": "dashboard_group_timer",
        "order": 1,
        "width": 0,
        "height": 0,
        "label": "Web 計時控制 LED",
```

```
        "labelPlacement": "left",
        "labelAlignment": "left",
        "colorForValue": [
            {
                "color": "#ff0000",
                "value": "false",
                "valueType": "bool"
            },
            {
                "color": "#00d000",
                "value": "true",
                "valueType": "bool"
            }
        ],
        "allowColorForValueInMessage": false,
        "shape": "circle",
        "showGlow": true,
        "name": "",
        "x": 530,
        "y": 220,
        "wires": []
    },
    {
        "id": "http_in_state_check",
        "type": "http in",
        "z": "flow_workspace",
        "name": "API 查詢狀態",
        "url": "/state",
        "method": "get",
        "upload": false,
        "swaggerDoc": "",
        "x": 130,
        "y": 300,
        "wires": [
            [
                "function_state_read"
            ]
        ]
    },
    {
        "id": "function_state_read",
        "type": "function",
        "z": "flow_workspace",
```

```
        "name": "讀取全域狀態",
        "func": "// 回傳目前記錄在系統內的燈號狀態\nmsg.payload = global.get(\"ledState\") ||
\"0\";\nreturn msg;",
        "outputs": 1,
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 320,
        "y": 300,
        "wires": [
            [
                "http_out_state_check"
            ]
        ]
    },
    {
        "id": "http_out_state_check",
        "type": "http response",
        "z": "flow_workspace",
        "name": "回傳即時狀態",
        "statusCode": "200",
        "headers": {},
        "x": 520,
        "y": 300,
        "wires": []
    },
    {
        "id": "dashboard_group_timer",
        "type": "ui_group",
        "name": "定時面板",
        "tab": "dashboard_tab_timer",
        "order": 1,
        "disp": true,
        "width": "6",
        "collapse": false
    },
    {
        "id": "dashboard_tab_timer",
        "type": "ui_tab",
        "name": "Web 定時器測試",
        "icon": "dashboard",
        "disabled": false
```

```
    "hidden": false
}
]
```

運作重點拆解

如果您想了解背後的邏輯，這幾個地方是關鍵魔法所在：

1. **Javascript 計時器 (setTimeout)**：在「計時與狀態控制」這個節點中，我們使用了原生的 setTimeout 函數。當收到 state=1 時，它會將秒數乘以 1000 轉換為毫秒，開始在 Node-RED 背景倒數。時間一到，它就會自動輸出 false 去關掉 ui_led 節點。
2. 防止重複觸發的防呆機制：如果您設定了 10 秒，但在第 5 秒又按了一次開關會發生什麼事？我在程式碼裡加入了 clearTimeout()，它會先砍掉舊的計時器，重新計算，確保行為穩定不會卡死。
3. 狀態同步 **(Polling)**：
   在「HTML 計時器網頁」的最下方，我加入了一段 setInterval 腳本，讓網頁每隔 **1** 秒鐘就去戳一次 **/state API**。一旦 Node-RED 的計時器跑完並將狀態設為 0，網頁端就會收到通知，自動把開關拉桿滑回 off，體驗就會非常流暢直覺！

部署完成後，只要前往 http://您的IP:1880/web，就可以拉動滑桿設定秒數，並享受自動定時關閉的效果了。